
HANDBOOK OF COMBINATORIAL OPTIMIZATION

Supplement Volume B

HANDBOOK OF COMBINATORIAL OPTIMIZATION

Supplement Volume B

Edited by

DING-ZHU DU

University of Minnesota, Minneapolis, MN

PANOS M. PARDALOS

University of Florida, Gainesville, FL

Springer

eBook ISBN: 0-387-23830-1
Print ISBN: 0-387-23829-8

©2005 Springer Science + Business Media, Inc.

Print ©2005 Springer Science + Business Media, Inc.
Boston

All rights reserved

No part of this eBook may be reproduced or transmitted in any form or by any means, electronic, mechanical, recording, or otherwise, without written consent from the Publisher

Created in the United States of America

Visit Springer's eBookstore at:
and the Springer Global Website Online at:

<http://ebooks.springerlink.com>
<http://www.springeronline.com>

Preface

Combinatorial (or discrete) optimization is one of the most active fields in the interface of operations research, computer science, and applied mathematics. Combinatorial optimization problems arise in various applications, including communications network design, VLSI design, machine vision, airline crew scheduling, corporate planning, computer-aided design and manufacturing, database query design, cellular telephone frequency assignment, constraint directed reasoning, and computational biology. Furthermore, combinatorial optimization problems occur in many diverse areas such as linear and integer programming, graph theory, artificial intelligence, and number theory. All these problems, when formulated mathematically as the minimization or maximization of a certain function defined on some domain, have a commonality of discreteness.

Historically, combinatorial optimization starts with linear programming. Linear programming has an entire range of important applications including production planning and distribution, personnel assignment, finance, allocation of economic resources, circuit simulation, and control systems. Leonid Kantorovich and Tjalling Koopmans received the Nobel Prize (1975) for their work on the optimal allocation of resources. Two important discoveries, the ellipsoid method (1979) and interior point approaches (1984) both provide polynomial time algorithms for linear programming. These algorithms have had a profound effect in combinatorial optimization. Many polynomial-time solvable combinatorial optimization problems are special cases of linear programming (e.g. matching and maximum flow). In addition, linear programming relaxations are often the basis for many approximation algorithms for solving NP-hard problems (e.g. dual heuristics).

Two other developments with a great effect on combinatorial optimization are the design of efficient integer programming software and the availability of parallel computers. In the last decade, the use of integer programming models has changed and increased dramatically. Two decades ago, only problems with up to 100 integer variables could be solved in a computer. Today we can solve problems to optimality with thousands of integer variables. Furthermore, we can compute provably good approximate solutions to problems with millions of integer variables. These advances have been made possible by developments in hardware, software and algorithm design.

The Handbooks of Combinatorial Optimization deal with several algorithmic approaches for discrete problems as well as with many combinatorial problems. We have tried to bring together almost every aspect of this enormous field with emphasis on recent developments. Each chapter in the Handbooks is essentially expository in nature, but of scholarly treatment.

The Handbooks of Combinatorial Optimization are addressed not only to researchers in discrete optimization, but to all scientists in various disciplines who use combinatorial optimization methods to model and solve problems. We are certain that experts in the field as well as nonspecialist readers will find the material of the Handbooks stimulating and helpful.

We would like to take this opportunity to thank the authors, the anonymous referees, and the publisher for helping us produce these volumes of the Handbooks of Combinatorial Optimization with state-of-the-art chapters. We would also like to thank Mr. Arnold Mayaka for making Author Index and Subject Index for this volume.

Ding-Zhu Du and Panos M. Pardalos

Contents

Preface	v
Data Correcting Algorithms in Combinatorial Optimization	1
<i>Diptesh Ghosh, Boris Goldengorin, and Gerard Sierksma</i>	
The Steiner Ratio of Banach-Minkowski Space - A Survey	55
<i>Dietmar Cieslik</i>	
Probabilistic Verification and Non-Approximability	83
<i>Mario Szegedy</i>	
Steiner Trees in Industry	193
<i>Xiuzhen Cheng, Yingshu Li, Ding-Zhu Du, and Hung Q. Ngo</i>	
Network-based Model and Algorithms in Data Mining and Knowledge Discovery	217
<i>Vladimir Boginski, Panos M. Pardalos, and Alkis Vazacopoulos</i>	
The Generalized Assignment Problem and Extensions	259
<i>Dolores Romero Morales and H. Edwin Romeijn</i>	
Optimal Rectangular Partitions	313
<i>Xiuzhen Cheng, Ding-Zhu Du, Joon-Mo Kim, and Lu Ruan</i>	
Connected Dominating Sets in Sensor Networks and MANETs	329
<i>Jeremy Blum, Min Ding, Andrew Thaeler, and Xiuzhen Cheng</i>	
Author Index	371
Subject Index	381

Data Correcting Algorithms in Combinatorial Optimization

Diptesh Ghosh

P&QM Area, Indian Institute of Management

Vastrapur, Ahmedabad 380015, Gujarat, India

E-mail: diptesh@iimahd.ernet.in

Boris Goldengorin

Faculty of Economic Sciences

University of Groningen, 9700AV Groningen, The Netherlands

E-mail: b.goldengorin@eco.rug.nl

Gerard Sierksma

Faculty of Economic Sciences

University of Groningen, 9700AV Groningen, The Netherlands

E-mail: g.sierksma@eco.rug.nl

Contents

1	Introduction	2
2	Data Correcting for Real-Valued Functions	3
3	Data Correcting for Combinatorial Optimization Problems	7
4	The Asymmetric Traveling Salesperson Problem	11
4.1	The Data Correcting Algorithm	11
4.2	Computational Experience with ATSP Instances	15
5	Maximization of General Submodular Functions	19
5.1	A Simple Data Correcting Algorithm	20
5.2	A Data Correcting Algorithm based on Multi-Level Search	22
5.3	Computational Experience with Quadratic Cost Partition Instances	29

6	The Simple Plant Location Problem	32
6.1	A Pseudo-Boolean Formulation of the SPLP	33
6.2	Preprocessing SPLP instances	36
6.3	The Data Correcting Algorithm	39
6.4	Computational Experience with SPLP Instances	41
6.4.1	Testing the Effectiveness of the Reduction Procedure RP . . .	41
6.4.2	Bilde and Krarup-type Instances	43
6.4.3	Galvão and Raggi-type Instances	44
6.4.4	Instances from the OR-Library	45
6.4.5	Körkel-type Instances with 65 Sites	46
6.4.6	Körkel-type Instances with 100 Sites	48

References

1 Introduction

Polynomially solvable special cases have long been studied in the literature on combinatorial optimization problems (see, for example, Gilmore *et al.* [19]). Apart from being mathematical curiosities, they often provide important insights for serious problem-solving. In fact, the concluding paragraph of Gilmore *et al.* [19] states the following, regarding polynomially solvable special cases for the traveling salesperson problem.

“... We believe, however, that in the long run the greatest importance of these special cases will be for approximation algorithms. Much remains to be done in this area.”

This chapter describes a step in the direction of incorporating polynomially solvable special cases into approximation algorithms. We review *data correcting algorithms* — approximation algorithms that make use of polynomially solvable special cases to arrive at high-quality solutions. The basic insight that leads to these algorithms is the fact that it is often easy to compute a bound on the difference between the costs of optimal solutions to two instances of a problem, even though it may be hard to compute optimal solutions for the two instances. These algorithms were first reported in the Russian literature (see Goldengorin [9, 10, 11, 12, 13]).

The approximation in data correcting algorithms is in terms of an *accuracy parameter*, which is an upper bound on the difference between the objective value of an optimal solution to the instance and that of a solution

returned by the data correcting algorithm. Note that this is *not* expressed as a fraction of the optimal objective value for this instance as in common ε -optimal algorithms but as actual deviations from the cost of optimal solutions.

Although we suggest the use of data correcting algorithms to solve NP-hard combinatorial optimization problems, they form a general problem solving tool and can be used for functions defined on a continuous domain as well. We will in fact, motivate the algorithm in the next section using a function defined on a continuous domain, and having a finite range. We then show in Section 3, how this approach can be adapted for combinatorial optimization problems. In the next three sections we describe actual implementation of data correcting algorithms to three problems, the asymmetric traveling salesperson problem, the maximization of a general submodular function, and the simple plant location problem.

2 Data Correcting for Real-Valued Functions

Consider a real-valued function $f : D \rightarrow \mathbb{R}$, where D is the domain on which the function is defined. We assume that f is not analytically tractable over D , but is computable in polynomial time for any $x \in D$, and concern ourselves with the problem of finding α -minimal solutions to the function f over D , i.e. the problem of finding a member of $\{x | x \in D, f(x) \leq f(x^*) + \alpha\}$, where $x^* \in \arg \min_D \{f(x)\}$, and α is the pre-defined *accuracy* parameter. The discussion here is for a minimization problem; the maximization problem can be dealt with in a similar manner.

Let us assume a partition $\{D_1, \dots, D_p\}$ of the domain D . Let us further assume that for each of the sub-domains D_i of D , $i = 1, \dots, p$, we are able to find functions $g_i : D_i \rightarrow \mathbb{R}$, which are easy to minimize over D_i , and such that

$$|f(x) - g_i(x)| \leq \frac{\alpha}{2} \quad \forall x \in D_i. \quad (1)$$

We call such easily minimizable functions *regular*.

Theorem 2.1 demonstrates an important relationship between the regular functions g_i and the original function f . It states that the function value of f at the best among the minima of all the g_i 's over their respective domains is close to the minimum function value of f over the domain.

Theorem 2.1 Let $x_i^\alpha \in \arg \min_{x \in D_i} \{g_i(x)\}$, $x^\alpha \in \arg \min_i \{f(x_i^\alpha)\}$, and let $x^* \in \arg \min_{x \in D} \{f(x)\}$. Then

$$f(x^\alpha) \leq f(x^*) + \alpha.$$

Proof: Let $x_i^* \in \arg \min_{x \in D_i} \{f(x)\}$. Then for $i = 1, \dots, p$, $f(x_i^\alpha) - \frac{\alpha}{2} \leq g_i(x_i^\alpha) \leq g_i(x_i^*) \leq f(x_i^*) + \frac{\alpha}{2}$, i.e. $f(x_i^\alpha) \leq f(x_i^*) + \alpha$. Thus $\min_i \{f(x_i^\alpha)\} \leq \min_i \{f(x_i^*)\} + \alpha$, which proves the result. ■

Notice that x^* and x^α do not need to be in the same sub-domain of D .

Theorem 2.1 forms the basis of the data correcting algorithm to find an approximate minimum of a function f over a certain domain D . The procedure consists of three steps: the first in which the domain D of the function is partitioned into several sub-domains; the second in which f is approximated in each of the sub-domains by regular functions following the condition in expression (1) and a minimum point of the regular function is obtained; and a third step, in which the minimum points computed in the second step are considered and the best among them is chosen as the output. This procedure can be further strengthened by using lower bounds to check if a given sub-domain can possibly lead to a solution better than any found thus far. The approximation of f by regular functions g_i is called *data correcting*, since an easy way of obtaining the regular functions is by altering the data that describe f . A pseudocode of the algorithm, which we call DC-G, is provided below.

Procedure DC-G(f, D, α)

Output: $x^\alpha \in D$ such that $f(x^\alpha) \leq \min\{f(x) | x \in D\} + \alpha$.

Code:

1. begin
2. create a partition $\{D_1, \dots, D_n\}$ of D ;
3. for each sub-domain D_i
4. begin
5. $\underline{f}_i$:= a lower bound to $f(x)$, $x \in D_i$;
6. if $\underline{f}_i \geq \text{bestvalue}$
7. continue;
8. construct a regular function $g_i(x)$ obeying (1);
9. $x_i^\alpha \in \arg \min_{x \in D_i} \{g_i(x)\}$;
10. end;
11. $\text{bestvalue} := \infty$;
12. if $f(x_i^\alpha) < \text{bestvalue}$;

```

13.    begin
14.         $x^\alpha := x_i^\alpha$ ;
15.         $bestvalue := f(x^\alpha)$ ;
16.    end;
17.    return  $x^\alpha$ ;
18. end.

```

Lines 5 through 7 in the code carry out the bounding process, and lines 8 and 9 implement the process of computing the minima of regular functions over each sub-domain. These steps are enclosed in a loop, so that at the end of line 10, all the minima of the regular functions are at hand. The code in lines 11 through 16 obtain the best among the minima obtained before. By Theorem 2.1, the solution chosen by the code in lines 11 through 16 is an α -minimum of f , and therefore, this solution is returned by the algorithm in line 17.

We will now illustrate the data correcting algorithm through an example. The example that we choose is one of a real-valued function of one variable, since these are some of the simplest functions to visualize.

Consider the problem of finding an α -minimum of the function f shown in Figure 1. The function is assumed to be well-defined, though analytically intractable on the domain D .

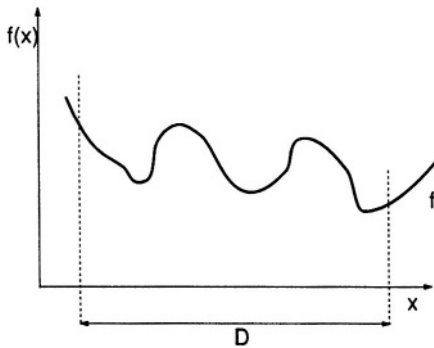


Figure 1: A general function f

The data correcting approach can be used to solve the problem above,

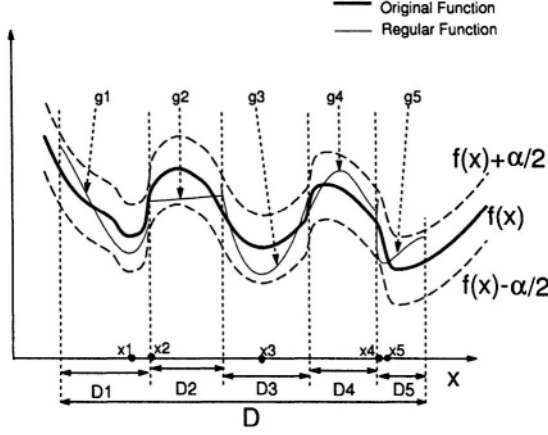


Figure 2: Illustrating the Data Correcting approach on f

i.e. of finding a solution $x^\alpha \in D$ such that $f(x^\alpha) \leq \min\{f(x)|x \in D\} + \alpha$.

Consider the partition $\{D1, D2, D3, D4, D5\}$ of D shown in Figure 2. Let us suppose that we have a regular function $g1(x)$ with $|g1(x) - f(x)| \leq \frac{\alpha}{2}$, $\forall x \in D1$. Assume also, that $x1$ is a minimum point of $g1(x)$ in $D1$. Since this is the best solution that we have so far, we store $x1$ as an α -minimal solution to $f(x)$ in the domain $D1$. We then consider the next interval in the partition, $D2$. We first obtain a lower bound on the minimum value of $f(x)$ on $D2$. If this bound is larger than $f(x1)$, we ignore this domain and examine domain $D3$. Let this not be the case in our example. So we construct a regular function $g2(x)$ with $|g2(x) - f(x)| \leq \frac{\alpha}{2}$, $\forall x \in D2$, and find $x2$, its minimum point over $D2$. Since $f(x2) \geq f(x1)$ (see Figure 2), we retain $x1$ as our α -optimal solution over $D1 \cup D2$. Proceeding in this manner, we examine $f(x)$ in $D3$ through $D5$, compute regular functions $g3(x)$ through $g5(x)$ for these domains, and compute $x3$ through $x5$. In this example, $x3$ replaces $x1$ as our α -minimal solution after consideration of $D3$, and remains so until the end. At the end of the algorithm, $x3$ is returned as a value of x^α .

There are four points worth noting at this stage. The first is that we need to examine all the sub-domains in the original domain before we re-

turn a near-optimal solution using this approach. The reason for this is very clear. The correctness of the algorithm depends on the result in Theorem 2.1, and this theorem only concerns the *best* among the minima of each of the sub-domains. For instance, in the previous example, if we stop as soon as we obtain the first α -optimal solution $\mathbf{x}_1$ we would be mistaken, since Theorem 2.1 applies to $\mathbf{x}_1$ only over $D_1 \cup D_2$. The second point is that there is no guarantee that the near-optimal solution returned by DC-G will be in the neighborhood of a true optimal solution. There is in fact, nothing preventing the near-optimal solution existing in a sub-domain different from the sub-domain of an optimal solution, as is evident from the previous example. The true minimum of f lies in the domain D_5 , but DC-G returns $\mathbf{x}_3$, which is in D_3 . The third point is that the regular functions $g_i(\mathbf{x})$ approximating $f(\mathbf{x})$ do not need to have the same functional form. For instance in Example 1, $g_1(\mathbf{x})$ is quadratic, while $g_2(\mathbf{x})$ is linear. Finally, for the proof of Theorem 2.1, it is sufficient for $\{D_1, \dots, D_n\}$ to be a cover of D (as opposed to a partition as required in the pseudocode of DC-G).

3 Data Correcting for Combinatorial Optimization Problems

The data correcting methodology described in the previous section can be incorporated into an implicit enumeration scheme (like branch and bound) and used to obtain near-optimal solutions to NP-hard combinatorial optimization problems. In this section we describe how this incorporation is achieved for a general combinatorial optimization problem.

We define a combinatorial optimization problem P as a collection of instances I . An instance I consists of a ground set $G = \{e_1, e_2, \dots, e_n\}$ of n elements, a cost vector $C_I = (c_1^I, c_2^I, \dots, c_n^I)$ corresponding to the elements in G , a set $S \subseteq 2^G$ of feasible solutions, and a cost function $f_I : S \rightarrow \mathbb{R}$. The objective is to obtain a solution, i.e. a member of S that minimizes the cost function. For example, for an asymmetric traveling salesperson problem (ATSP) instance on a digraph $G = (V, A)$, with a distance matrix $D = [d_{ij}]$, we have $G = A$, $c_{ij}^I = d_{ij}$, S is the set of all Hamiltonian cycles in G , and $f_I(s) = \sum_{(i,j) \in s} d_{ij}$ for each $s \in S$.

Implicit enumeration algorithms for combinatorial problems include two main strategies, namely branching and fathoming. Branching involves partitioning the set of feasible solutions S into smaller subsets. This is done under the assumption that optimizing the cost function over a more restricted so-

lution space is easier than optimizing it over the whole space. Fathoming involves one of two processes. First, we could compute lower bounds to the value that the cost function can attain over a particular member of the partition. If this bound is not better than the best solution found thus far, the corresponding subset in the partition is ignored in the search for an optimal solution. The second method of fathoming is by computing the optimum value of the cost function over the particular subset of the solution space (if that can be easily computed for the particular subset). We see therefore that two of the main requirements of the data correcting algorithm presented in the previous section, i.e. partitioning and bounding, are automatically taken care of for combinatorial optimization problems by implicit enumeration. The only other requirement that we need to consider is that of obtaining regular functions approximating f_I over subsets of the solution space.

Notice that the cost function $f_I(s)$ is a function of the cost vector C . So if the values of the entries in C are changed, $f_I(s)$ undergoes a change as well. Therefore, cost functions corresponding to polynomially solvable special cases can be used as “regular functions” for combinatorial optimization problems. Also note that for the same reason, the accuracy parameter can be compared with a suitably defined distance measure between two cost vectors, (or equivalently, two instances). Consider a subproblem in the tree obtained by normal implicit enumeration. The problem instance that is being evaluated at that subproblem is a restricted version of the original problem instance, i.e., it evaluates the cost function of the original problem instance for a subset S_k of the original solution space S . If we alter the data of the problem instance in a way that the altered data corresponds to a polynomially solvable special case, while guaranteeing that the cost of an optimal solution to the altered problem in S_k is not more than an acceptable amount higher than the cost of an optimal solution to original instance in S_k , then the altered cost function can be considered to be a regular approximation of the cost function of the original instance in S_k .

For combinatorial optimization problems, let us define a *proximity measure* $\rho(I_1, I_2)$ between two problem instances I_1 and I_2 , as an upper bound for the difference between $f_{I_1}(s_1^*)$ and $f_{I_1}(s_2^*)$, where s_1^* and s_2^* are optimal solutions to I_1 and I_2 respectively. The following lemma shows that the *Hamming distance* between the cost vectors of the two instances is a proximity measure when the cost function is of the sum type or the max type.

Lemma 3.1 *If the cost function of an instance I of a combinatorial optimization problem is of the sum type, (i.e. $f_I(s) = \sum_{e_k \in s} c_k^I$) or the max type, (i.e. $f_I(s) = \max_{e_k \in s} c_k^I$) then the measure*

$$\rho(I_1, I_2) = \sum_{e_i \in G} |c_i^{I_1} - c_i^{I_2}| \quad (2)$$

between two instances I_1 and I_2 of the problem is an upper bound to the difference between $f_{I_1}(s_1^)$ and $f_{I_1}(s_2^*)$, where s_1^* and s_2^* are optimal solutions to I_1 and I_2 , respectively.*

Proof: We will prove the result for sum type cost functions. The proof for max type cost functions is similar.

For sum type cost functions, it is sufficient to prove the result when the cost vectors C_{I_1} and C_{I_2} differ in only one position. Let $c_k^{I_1} = c_k^{I_2}$ for $k = 1, 2, \dots, j-1, j+1, \dots, n$, and $c_j^{I_1} \neq c_j^{I_2}$. Consider any solution $s \in S$. There are two cases to consider:

- $e_j \in s$: In this case, $|f_{I_1}(s) - f_{I_2}(s)| = \sum_{e_k \in s} |c_k^{I_1} - c_k^{I_2}| = |c_j^{I_1} - c_j^{I_2}|$.
- $e_j \notin s$: In this case it is clear that $f_{I_1}(s) = f_{I_2}(s)$.

Therefore, $|f_{I_1}(s) - f_{I_2}(s)| \leq \sum_{e_i \in G} |c_i^{I_1} - c_i^{I_2}| = \rho(I_1, I_2)$ for any solution $s \in S$, which automatically implies that $\rho(I_1, I_2)$ as defined in the statement of Lemma 3.1 is an upper bound for the difference between $f_{I_1}(s_1^*)$ and $f_{I_1}(s_2^*)$, where s_1^* and s_2^* are optimal solutions to I_1 and I_2 , respectively. ■

At this point, it is important to point out the difference between a fathoming step and a data correcting step. The bounds used in fathoming steps consistently overestimate the objective function of optimal solutions in maximization problems and underestimate it for minimization problems. The amount of over- or underestimation is not bounded. In data correcting steps however, the “regular function” may overestimate or underestimate the objective function, regardless of the objective of the problem. However, there is always a bound on the deviation of the “regular function” from the objective function of the problem.

One way of implementing the data correcting for a NP-hard problem instance I is the following. We first execute a data correcting step. We construct a polynomially solvable relaxation I_L of the original instance, and obtain an optimal solution x_L to I_L . Note that x_L need not be feasible to I . We next construct the best solution x to I that we can, starting from

x_L . (If such a solution is not possible, we conclude that the instance does not admit a feasible solution.) We also construct an instance I_C of the problem, which will have x as an optimal solution. The proximity measure $\rho(I, I_C)$ then is an upper bound to the difference between the costs of x and of an optimal solution to I . I_C is called a *correction* of the instance I . If the proximity measure is not more than the allowable accuracy, then we can output x as an adequate solution to I . If this is not the case, then we partition the feasible solution space of I (these are formed by adding constraints to the formulation of I) and apply the data correction step to each of these subproblems.

The similarity in the procedural aspects of the data correcting step described above (and illustrated in the example) to fathoming rules used in branch and bound implementations makes it convenient to incorporate data correcting in the framework of implicit enumeration. We present the pseudocode of a recursive version of branch and bound incorporating data correcting below. The initial input to this procedure is the data for the original instance I , the feasible solution set S , any solution $s \in S$, and the accuracy parameter α . Notice that the data correcting step discussed earlier in this section is implemented in lines 6 through 10 in the pseudocode.

Algorithm DC(I, S, α)

Output: $x^\alpha \in S$ such that $f_I(x^\alpha) \leq \min\{f_I(x) | x \in S\} + \alpha$.

Code:

```

1. begin
2.    $s :=$  a solution in  $S$ ;
3.    $lb :=$  a lower bound on the value of  $f_I(x)$  over  $S$ ;
4.   if  $f_I(s) = lb$  return  $s$ ;
5.   compute an optimal solution  $s_L$  to a polynomially solvable
6.   relaxation  $I_L$  to  $I$ ;
7.   if possible then construct a solution  $s$  to  $I$  starting from  $s_L$ ;
8.   else return "infeasible";          (* No solution  $s$  can be constructed *)
9.   construct an instance  $I_C$  that has  $s$  as an optimal solution;
10.  if  $\rho(I_C, I) \leq \alpha$ 
11.    return  $s$ ;
12.  else begin
13.    partition  $S$  into subsets  $S_1$  through  $S_n$ ;
14.    for  $i := 1$  to  $n$                       (* Branch *)
15.       $s_i :=$  DC( $I, S_i, \alpha$ );
```

```

16.          return the best solution from among  $s_1$  through  $s_n$ ;
17.      end;
18. end.

```

The algorithm described above is a prototype. We have not specified how the lower bound is to be computed, or which solution to choose in the feasible region, or how to partition the domain into sub-domains. These are details that vary from problem to problem, and are an important part in the engineering aspects of the algorithm. Note that this is just one of many possible ways of implementing a data correcting algorithm.

We can now describe our implementation of data correcting on specific combinatorial optimization problems. The next section deals with asymmetric traveling salesperson problems. The implementation of the data correcting algorithm for this problem closely follows the pseudocode above. Sections 5 and 6 deal with the maximization of general submodular functions and the simple plant location problem. Our implementations of data correcting for these two problems are slightly different, in which the data correction is done in an implicit manner.

4 The Asymmetric Traveling Salesperson Problem

In an asymmetric traveling salesperson problem (ATSP) instance we are given a weighted digraph $G = (V, A)$ and a $|V| \times |V|$ distance matrix $D = [d_{ij}]$ and our objective is output a least cost Hamiltonian cycle in this graph. This is one of the most studied problems in combinatorial optimization, see Lawler *et al.* [26] and Gutin and Punnen [20] for a detailed introduction.

4.1 The Data Correcting Algorithm

The data correcting algorithm (DC) presented in the previous section can be easily mapped to the ATSP. Lemma 3.1 takes the following form for ATSP instances.

Lemma 4.1 *Consider two ATSP instances I_1 and I_2 defined on digraphs $G_1 = (V_1, A_1)$ and $G_2 = (V_2, A_2)$ respectively, with $|V_1| = |V_2|$. Let $D_1 = [d_{ij}^1]$ and $D_2 = [d_{ij}^2]$ be the distance matrices associated with I_1 and I_2 .*

Further let T_1 and T_2 be optimal solutions to I_1 and I_2 , and let L_1 and L_2 respectively represent the lengths of T_1 and T_2 in instance I_1 . Then

$$L_2 - L_1 \leq \sum_{(i,j) \in A} |d_{ij}^1 - d_{ij}^2|.$$

Before presenting a pseudocode for the algorithm, let us illustrate the data correcting step for the ATSP with an example. Consider the 6-city ATSP instance with the distance matrix $D = [d_{ij}]$ shown below. (This corresponds to I in the discussion in the previous section.)

D	1	2	3	4	5	6
1	-	10	16	19	25	22
2	19	-	10	13	13	10
3	10	28	-	22	16	13
4	19	25	13	-	10	19
5	16	22	19	13	-	11
6	13	22	15	13	10	-

If we allow subtours in solutions to the ATSP, we get the assignment problem relaxation. Solving the assignment problem on D , using the Hungarian method, we get the following reduced distance matrix $D^H = [d_{ij}^H]$. (The assignment problem with the distance matrix D corresponds to I_L .)

D^H	1	2	3	4	5	6
1	-	0	6	7	16	12
2	9	-	0	1	4	0
3	0	18	-	10	7	3
4	8	14	2	-	0	8
5	5	11	8	0	-	0
6	2	11	4	0	0	-

This leads to a solution with two cycles (1231) and (4564) (corresponding to x_L). Using patching techniques (see for example Lawler *et al.* [26]), we obtain a solution (1245631) (corresponding to x). Notice that (1245631) would be an optimal solution to the assignment problem if d_{24}^H and d_{63}^H had been set to zero in D^H , and that would have been the situation, if d_{24} and d_{63} were initially reduced by 4 and 1 respectively, i.e. if the distance matrix in the original ATSP instance was D^P defined below. (This corresponds to I_C .)

D^P	1	2	3	4	5	6
1	-	10	16	19	25	22
2	19	-	10	9	13	10
3	10	28	-	22	16	13
4	19	25	13	-	10	19
5	16	22	19	13	-	11
6	13	22	14	13	10	-

Therefore D^P is the distance matrix of the correction of the instance with distance matrix D . The proximity measure $\rho(D, D^P) = \sum_{i=1}^6 \sum_{j=1}^6 |d_{ij} - d_{ij}^P| = |d_{24} - d_{24}^P| + |d_{63} - d_{63}^P| = 4 + 1 = 5$.

A proximity measure is an upper bound to the difference between the costs of two solutions for a problem instance, so the stronger the bound, the better would be the performance of any enumeration algorithm dependent on such bounds. It is possible to obtain stronger performance measures for ATSP instances, for example

$$\rho_1(D, D^P) = \min \left\{ \sum_{i=1}^n \max_{1 \leq j \leq n} |d_{ij} - d_{ij}^P|, \sum_{j=1}^n \max_{1 \leq i \leq n} |d_{ij} - d_{ij}^P| \right\} \quad (3)$$

is a better proximity measure than the one defined in (2).

Given the data correcting step, the DC algorithm presented in the previous section can be modified to solve ATSP instances. The pseudocode for a recursive version of this algorithm is given below.

Algorithm DCA-ATSP(G, α)

Output: A tour x^α such that the difference between the cost of x^α and the cost of an optimal tour is not more than α

Code:

1. begin
2. $s :=$ an arbitrary tour in G ;
3. $lb :=$ a lower bound on the cost of an optimal tour;
4. if $f_I(s) = lb$ return s ;
5. $s_L :=$ an optimal solution to the assignment problem on G ;
6. construct a solution s from s_L through patching;
7. using s_L , compute the proximity measure ρ ;
8. if $\rho \leq \alpha$
9. return s ;

```

10.      else begin                                     (* Branch *)
11.          branch according to a pre-decided branching rule;
12.          for each subproblem  $i$  generated
13.               $s_i :=$  the solution output by DCA-ATSP on subproblem  $i$ ;
14.          return the best solution from among all  $s_i$ 's;
15.      end;
16. end.

```

Note that a good lower bound can be incorporated into DCA-ATSP to make it more efficient.

We next illustrate the DCA-ATSP algorithm above on an instance of the ATSP. Consider the 8-city ATSP instance with the distance matrix $D = [d_{ij}]$ shown below. (This example was taken from Balas and Toth [1], p. 381).

D	1	2	3	4	5	6	7	8
1	-	2	11	10	8	7	6	5
2	6	-	1	8	8	4	6	7
3	5	12	-	11	8	12	3	11
4	11	9	10	-	1	9	8	10
5	11	11	9	4	-	2	10	9
6	12	8	5	2	11	-	11	9
7	10	11	12	10	9	12	-	3
8	7	10	10	10	6	3	1	-

We use

- the proximity measure ρ (see Expression (2)) for data correction,
- the assignment algorithm to compute lower bounds for subproblems,
- a patching algorithm to create feasible solutions, and compute proximity measures, and
- the patched solution derived from the assignment solution as a feasible solution in the domain.

The branching rule used in this example is as follows. At each subproblem, we construct the assignment problem solution and then patch it. We also correct the original matrix to a new matrix that would output the patched solution if the assignment problem is solved on it. We next identify

the arc corresponding to the entry in the new matrix that had to be corrected by the maximum amount. The tail of this arc is identified, and we branch on all the arcs in the subtour containing that vertex. For example, in this problem, the assignment solution is (1231)(4564)(787), which is patched to (123786451) and the cost of patching is 9. If we correct the problem data, we will see that the entry d_{51} (corresponding to arc (5,1)) contributes the maximum amount (7) to the patching. Hence we branch on each arc in the cycle (4564), and construct three subproblems, the first with the additional constraint that arc (4,5) be excluded from the solution, the second with the additional constraint that arc (5,6) be excluded from the solution, and the third with the additional constraint that arc (4,5) be excluded from the solution.

The polynomially solvable special case that we consider is the set of all ATSP instances for which the assignment procedure gives rise to a cyclic permutation.

Using the branching rule described above, depth-first branch and bound generates the enumeration tree of Figure 3. The nodes are labeled according to the order in which the problems at the corresponding nodes were evaluated.

Since the cost of patching equals the value of ρ , we can now evaluate the performance of data correcting on this example. If the allowable accuracy parameter α is set to 0, then the enumeration tree constructed by DC will be the one shown in Figure 3 and evaluates 8 subproblems. However if the value of α is set to 1, then enumeration stops after node 4, since the lower bound obtained is 25 which is one less than the solution we have at hand.

The previous example shows that the data correcting algorithm can be a very attractive alternative to branch and bound algorithms. In the next subsection we report experiences of the performance of the data correcting algorithm on ATSP instances from the TSPLIB [30].

4.2 Computational Experience with ATSP Instances

TSPLIB has twenty seven ATSP instances, out of which we have chosen twelve for our experiments. These twelve can be solved to optimality within five hours using an ordinary branch and bound algorithm. Eight of these belong to the 'ftv' class of instances, while four belong to the 'rbg' class. We implemented DCA-ATSP in C and ran it on a Intel Pentium based computer running at 666MHz with 128MB RAM.

The results of our experiments are presented graphically in Figures 4

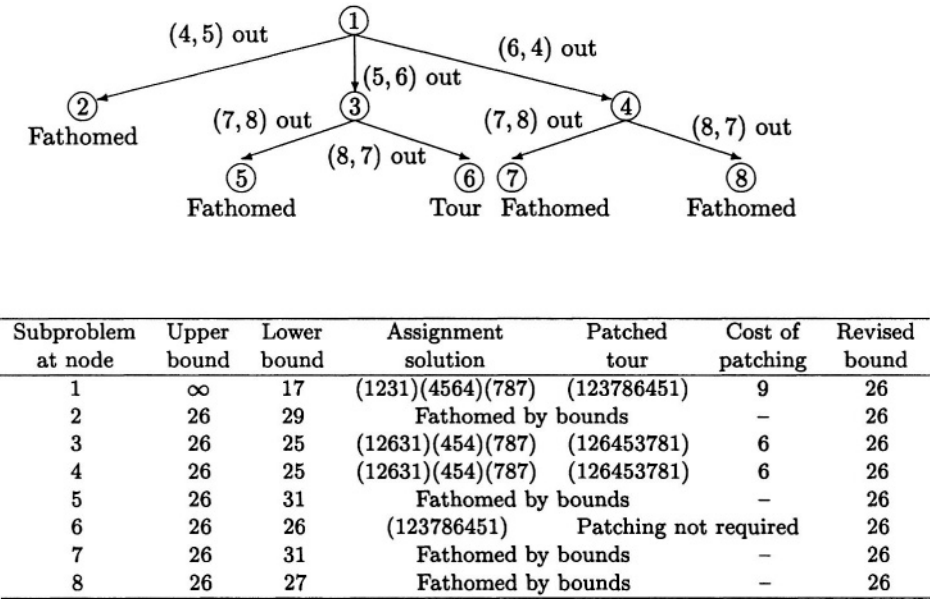


Figure 3: Branch and bound tree for the instance in the example.

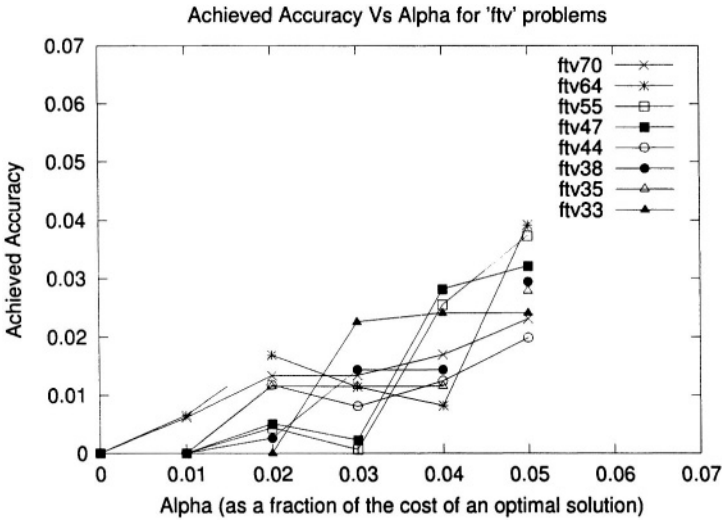


Figure 4: Accuracy achieved versus α for *ftv* instances

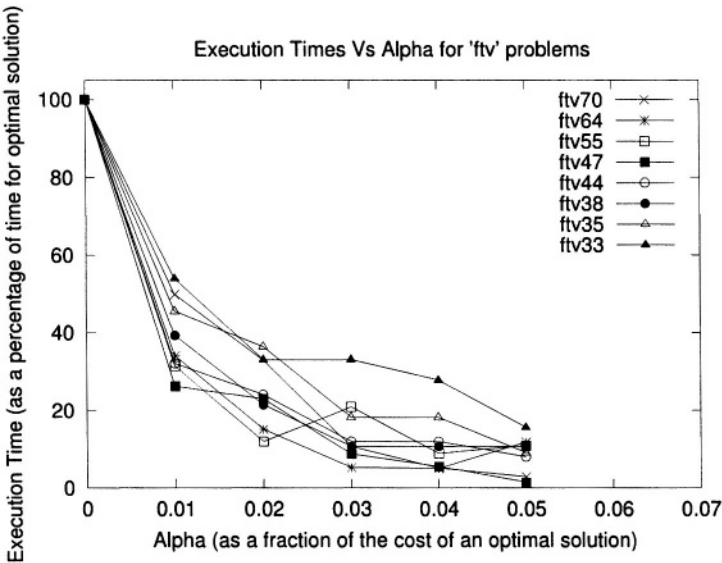


Figure 5: Variation of execution times versus α for *ftv* instances

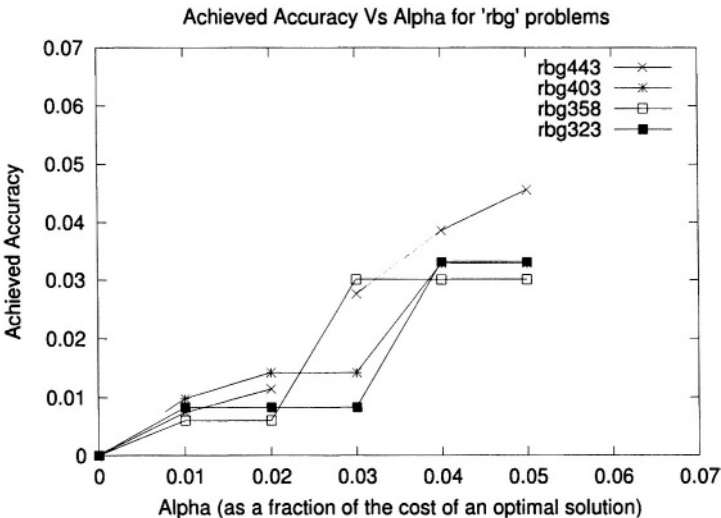


Figure 6: Accuracy achieved versus α for *rbg* instances

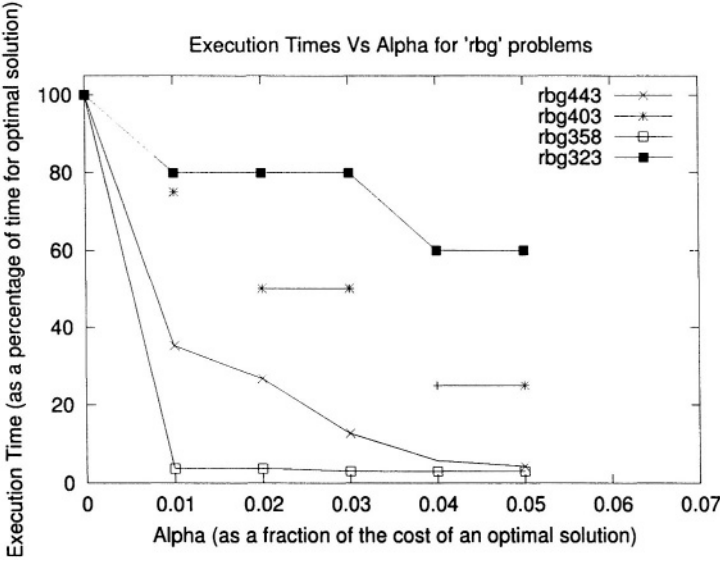


Figure 7: Variation of execution times versus α for *rbg* instances

through 7. In computing accuracies, (Figures 4 and 6) we have plotted the accuracy and deviation of the solution output by the data correcting algorithm from the optimal (called ‘achieved accuracy’ in the figures) as a fraction of the cost of an optimal solution to the instance. We observed that for each of the twelve instances that we studied, the achieved accuracy is consistently less than 80% of the pre-specified accuracy.

There was a wide variation in the CPU time required to solve the different instances. For instance, *ftv70* required 17206 seconds to solve to optimality, while *rbg323* required just 5 seconds. Thus, in order to maintain uniformity while demonstrating the variation in execution times with respect to changes in α values, we represented the execution times for each instance for each α value as a percentage of the execution time required to solve that instance to optimality. Notice that for all the *ftv* instances when α was 5% of the cost of the optimal solution, the execution time reduced to 20% of that required to solve the respective instance to optimality. The reduction in execution times for *rbg* instance was equally steep, with the exception of *rbg323* which was in any case an easy instance to solve.

In summary, it is quite clear that data correcting is an effective methodology for solving ATSP instances. There are usually steep reductions in

execution times even when the allowed accuracy is very small. This makes the method very useful for solving real world problems where a near-optimal solution is often acceptable provided the execution times are not too long.

5 Maximization of General Submodular Functions

Let $N = \{1, 2, \dots, n\}$ and 2^N denote the set of all subsets of N . A function $z : 2^N \rightarrow \mathbb{R}$ is called *submodular* if for each $I, J \in 2^N$, $z(I) + z(J) \geq z(I \cup J) + z(I \cap J)$. The solution process of many classical combinatorial optimization problems, like the generalized transportation problem, the quadratic cost partition (QCP) problem with nonnegative edge weights, and set covering, can be formulated as the maximization of a submodular function (MSF), i.e. the problem:

$$\max\{z(I) | I \subseteq N\}.$$

Although the general problem of the maximization of a submodular function is known to be NP-hard (see Lovasz [28]), there has been a sustained research effort aimed at developing practical procedures for solving medium and large-scale problems in this class. In the remainder of this section we suggest two data correcting algorithms for solving the problem. Note that Lemma 3.1 assumes the following form for this problem.

Lemma 5.1 *Consider two submodular functions $z_1(\bar{x}) = \sum_{i=1}^n \sum_{j=i}^n c_{ij}^1 \prod_{k=i}^j x_k$ and $z_2(\bar{x}) = \sum_{i=1}^n \sum_{j=i}^n c_{ij}^2 \prod_{k=i}^j x_k$. Let $\bar{x}_1^*$ and $\bar{x}_2^*$ be the maximum points of $z_1(\bar{x})$ and $z_2(\bar{x})$ respectively. Then*

$$z_1(\bar{x}_1^*) - z_1(\bar{x}_2^*) \leq \sum_{i=1}^n \sum_{j=1}^n |c_{ij}^1 - c_{ij}^2|.$$

The algorithm described in Section 5.1 have been published in Goldengorin *et al.* [14] while that described in Section 5.2 have been published in Goldengorin and Ghosh [18]. For each of the two algorithms in we first describe a class of polynomially solvable instances for submodular function maximization problems. We then describe the data correcting algorithms that uses this class of polynomially solvable instances to solve a general submodular function maximization problem. The classes of polynomially solvable instances are algorithmically defined, i.e. they are classes of instances that are solved to optimality using a pre-specified polynomial algorithm.

5.1 A Simple Data Correcting Algorithm

The class of polynomially solvable instances that we describe here is defined using a polynomial time algorithm called the Preliminary Preservation (PP) algorithm. Normally these algorithms terminate with a subgraph of the Hasse diagram of the original instance which is guaranteed to contain the maximum. However, for instances where PP returns a subgraph with a single node, that node *is* the maximum, and the instance is said to have been solved in polynomial time. Instances such as these make up the class of polynomially solvable instances that we consider here.

Let z be a real-valued function defined on the power set 2^N of $N = \{1, 2, \dots, n\}$; $n \geq 1$. For each $S, T \in 2^N$ with $S \subseteq T$, we define

$$[S, T] = \{I \in 2^N \mid S \subseteq I \subseteq T\}.$$

Note that $[\emptyset, N] = 2^N$. Any *interval* $[S, T]$ is a subinterval of $[\emptyset, N]$ if $\emptyset \subseteq S \subseteq T \subseteq N$. We denote this using the notation $[S, T] \subseteq [\emptyset, N]$. In this section an interval is always a subinterval of $[\emptyset, N]$. It is assumed that z attains a finite maximum value on $[\emptyset, N]$ which is denoted by $z^*[\emptyset, N]$, and $z^*[S, T] = \max\{z(I) \mid I \in [S, T]\}$ for any $[S, T] \subseteq [\emptyset, N]$. We also define $d_k^+(I) = z(I + k) - z(I)$ and $d_k^-(I) = z(I - k) - z(I)$.

The following theorem and corollaries from Goldengorin *et al.* [14] act as a basis for the Preliminary Preservation (PP) algorithm described therein.

Theorem 5.2 *Let z be a submodular function on $[S, T] \subseteq [\emptyset, N]$ and let $k \in T \setminus S$. Then the following assertions hold.*

1. $z^*[S + k, T] - z^*[S, T - k] \leq z(S + k) - z(S) = d_k^+(S)$.
2. $z^*[S, T - k] - z^*[S + k, T] \leq z(T - k) - z(T) = d_k^-(T)$.

Corollary 5.3 *(Preservation rules of order zero). Let z be a submodular function on $[S, T] \subseteq [\emptyset, N]$, and let $k \in T \setminus S$. Then the following assertions hold.*

1. *First Preservation Rule: If $d_k^+(S) \leq 0$, then $z^*[S, T] = z^*[S, T - k] \geq z^*[S + k, T]$.*
2. *Second Preservation Rule: If $d_k^-(T) \leq 0$, then $z^*[S, T] = z^*[S + k, T] \geq z^*[S, T - k]$.*

The PP algorithm accepts an interval $[S, T]$, $S \subseteq T$ and tries to apply Corollary 5.3 repeatedly. It returns an interval $[X, Y]$, $S \subseteq X \subseteq Y \subseteq T$, such that $z^*[S, T] = z^*[X, Y]$. The pseudocode for this algorithm is given below.

Algorithm PP $([S, T])$

Output: A subinterval of $[S, T]$ containing the maximum of z over $[S, T]$.

Code:

```

1. begin
2.   if  $T = S$  return  $[S, S]$ ;
3.   while  $T \neq S$  do begin
4.      $d_{max}^+ := \max\{d_k^+ | k \in T \setminus S\}$ ;
5.      $d_{max}^- := \max\{d_k^- | k \in T \setminus S\}$ ;
6.     if  $d_{max}^+ \leq 0$  then begin
7.        $k_{max}^+ := \arg \min\{k \in T \setminus S | d_k^+ = d_{max}^+\}$ ;
8.        $T := T - k_{max}^+$ ;
9.     end
10.    else if  $d_{max}^- \leq 0$  then begin
11.       $k_{max}^- := \arg \min\{k \in T \setminus S | d_k^- = d_{max}^-\}$ ;
12.       $S := S + k_{max}^-$ ;
13.    end
14.    else return  $[S, T]$ ;
15.  end;
16. end.
```

The PP algorithm is called repeatedly by the DCA-MSF to generate a solution to the MSF instance within the prescribed accuracy level α . The pseudocode for DCA-MSF is given below. As in the case of ATSP, a good problem-specific upper bound will improve the performance of the algorithm.

Algorithm DCA-MSF $([S, T], \alpha)$

Output: $x^\alpha \in [S, T]$ such that $z(x^\alpha) \geq z^*[S, T] - \alpha$.

Code:

```

1. begin
2.    $[S, T] := \text{PP}([S, T])$ ;
```

```

3.   if  $T = S$  return  $S$ ;
4.    $d_{max}^+ := \max\{d_k^+ | k \in T \setminus S\}$ ;
5.    $d_{max}^- := \max\{d_k^- | k \in T \setminus S\}$ ;
6.   if  $d_{max}^+ \leq d_{max}^-$  then begin
7.       if  $d_{max}^+ \leq \alpha$  then begin
8.            $k_{max}^+ := \arg \min\{k \in T \setminus S | d_k^+ = d_{max}^+\}$ ;
9.           return DCA-MSF( $[S, T - k_{max}^+]$ ,  $\alpha - d_{max}^+$ ) (* Correction *)
10.      end;
11.      else begin (* Branch *)
12.           $x_1 := \text{DCA-MSF}([S + k_{max}^+, T], \alpha)$ ;
13.           $x_2 := \text{DCA-MSF}([S, T - k_{max}^+], \alpha)$ ;
14.          if  $z(x_1) \geq z(x_2)$  return  $x_1$ 
15.          else return  $x_2$ ;
16.      end;
17.   end
18.   else begin
19.       if  $d_{max}^- \leq \alpha$  then begin
20.            $k_{max}^- := \arg \min\{k \in T \setminus S | d_k^- = d_{max}^-\}$ ;
21.           return DCA-MSF( $[S + k_{max}^-, T]$ ,  $\alpha - d_{max}^-$ ) (* Correction*)
22.       end;
23.       else begin (* Branch *)
24.            $x_1 := \text{DCA-MSF}([S + k_{max}^-, T], \alpha)$ ;
25.            $x_2 := \text{DCA-MSF}([S, T - k_{max}^-], \alpha)$ ;
26.           if  $z(x_1) \geq z(x_2)$  return  $x_1$ 
27.           else return  $x_2$ ;
28.       end;
29.   end;
30. end.

```

5.2 A Data Correcting Algorithm based on Multi-Level Search

The preservation rules mentioned in Corollary 5.3 look at a level which is exactly one level deeper in the Hasse diagram than the levels of S and T . However, instead of looking one level deep we may look r levels deep in order to determine whether we can include or exclude an element. Let

$$M_r^+[S, T] = \{I \in [S, T] | |I \setminus S| \leq r\},$$

$$M_r^-[S, T] = \{I \in [S, T] \mid |T \setminus I| \leq r\}.$$

The set $M_r^+[S, T]$ is a collection of all sets representing solutions containing more elements than S , and which are no more than r levels deeper than S in the Hasse diagram. Similarly, the set $M_r^-[S, T]$ is a collection of all sets representing solutions containing less elements than T , and which are no more than r levels deeper than T in the Hasse diagram. Let us further define the collections of sets

$$\begin{aligned} N_r^+[S, T] &= M_r^+[S, T] \setminus M_{r-1}^+[S, T], \\ N_r^-[S, T] &= M_r^-[S, T] \setminus M_{r-1}^-[S, T]. \end{aligned}$$

The sets $N_r^+[S, T]$ and $N_r^-[S, T]$ are the collection of sets which are located exactly r levels above S and below T in the Hasse diagram, respectively.

Further, let $v_r^+[S, T] = \max\{z(I) \mid I \in M_r^+[S, T]\}$, $v_r^-[S, T] = \max\{z(I) \mid I \in M_r^-[S, T]\}$, $w_{r,k}^+[S, T] = \max\{d_t^+(I) \mid I \in N_r^+[S + k, T]\}$ and $w_{r,k}^-[S, T] = \max\{d_t^-(I) \mid I \in N_r^-[S, T - k]\}$.

Theorem 5.4 *Let z be a submodular function on $[S, T] \subseteq [\emptyset, N]$ with $k \in T \setminus S$ and let r be a positive integer. Then the following assertions hold.*

1. *If $|N_r^+[S + k, T]| > 0$, then $z^*[S + k, T] - \max\{z^*[S, T - k], v_r^+[S, T]\} \leq \max\{w_{r,k}^+[S, T], 0\}$.*
2. *If $|N_r^-[S, T - k]| > 0$, then $z^*[S, T - k] - \max\{z^*[S + k, T], v_r^-[S, T]\} \leq \max\{w_{r,k}^-[S, T], 0\}$.*

Proof: We prove only part 1 since the proof of the part 2 is similar. We may represent the partition of interval $[S, T]$ as follows:

$$[S, T] = M_r^+[S, T] \cup \bigcup_{I \in N_r^+[S, T]} [I, T].$$

Using this representation on the interval $[S + k, T]$, we have $z^*[S + k, T] = \max\{v_r^+[S + k, T], \max\{z^*[I + k, T] \mid I \in N_r^+[S, T]\}\}$. Let $I(k) \in \arg \max\{z^*[I + k, T] \mid I \in N_r^+[S, T]\}$.

There are two cases to consider: $z^*[I(k) + k, T] \geq v_r^+[S + k, T]$, and $z^*[I(k) + k, T] < v_r^+[S + k, T]$.

In the first case $z^*[S + k, T] = z^*[I(k) + k, T]$. For $I(k) \in N_r^+[S, T]$ we can apply Theorem 5.2(a) on the interval $[I(k), T]$ to obtain $z^*[I(k) + k, T] - z^*[I(k), T - k] \leq d_k^+(I(k))$, so that in this case $z^*[S + k, T] - z^*[I(k), T - k] \leq d_k^+(I(k))$. Note that for $[I(k), T - k] \subseteq [S, T - k]$ we have $z^*[S, T - k] \geq z^*[I(k), T - k]$, which implies that $z^*[S + k, T] - z^*[S, T - k] \leq d_k^+(I(k))$. Adding two maximum operations we get

$$z^*[S + k, T] - \max\{z^*[S, T - k], v_r^+[S + k, T]\} \leq \max\{d_k^+(I(k)), 0\}.$$

Since $w_{rk}^+[S, T]$ is the maximum of $d_k^+(I)$ for $I \in N_r^+[S + k, T]$, we have the required result.

In the second case $z^*[S + k, T] = v_r^+[S + k, T]$ which implies that $z^*[S + k, T] - v_r^+[S + k, T] = 0$ or $z^*[S + k, T] - \max\{z^*[S, T - k], v_r^+[S + k, T]\} \leq 0$. Adding a maximum operation with $w_{rk}^+[S, T]$ completes the proof. ■

Corollary 5.5 (*Preservation rules of order r*). *Let z be a submodular function on $[S, T] \subseteq [\emptyset, N]$ and let $k \in T \setminus S$. Then the following assertions hold.*

1. *First Preservation Rule of Order r : If $w_{rk}^+[S, T] \leq 0$, then $z^*[S, T] = \max\{z^*[S, T - k], v_r^+[S + k, T]\} \geq z^*[S + k, T]$.*
2. *Second Preservation Rule of Order r : If $w_{rk}^-[S, T] \leq 0$, then $z^*[S, T] = \max\{z^*[S + k, T], v_r^-[S, T - k]\} \geq z^*[S, T - k]$.*

Notice that when we apply Corollary 5.3 to an interval, we get a reduced interval, however, when we apply Corollary 5.5, we get a value v_r in addition to a reduced interval.

It can be proved by induction that the portion of the Hasse diagram eliminated by preservation rules of order $r - 1$ while searching for a maximum of the submodular function will certainly be eliminated by preservation rules of order r . In this sense, preservation rules of order r are not weaker than preservation rules of order $r - 1$. (A detailed proof for the result that preservation rules of order 1 are not weaker than preservation rules of order 0, refer to Goldengorin [17]).

In order to apply Corollary 5.5, we need functions that compute the value of $w_{rk}^+[S, T]$, $w_{rk}^-[S, T]$, $v_r^+[S + k, T]$, and $v_r^-[S, T - k]$. To that end, we define two recursive functions, *PPArplus* to compute $w_{rk}^+[S, T]$ and $v_r^+[S + k, T]$, and *PPArminus* to compute $w_{rk}^-[S, T]$ and $v_r^-[S, T - k]$. The pseudocode for *PPArplus* is shown below. Its output is a 3-tuple, containing, in order,